

Lab 3 — The Command Line, and git

How you'll submit everything for the rest of the semester

Amir Farbin

Today

I'm not here — read this first

[Watch the Lab 3 video](#) · *20 min*

Or: **Teams** → **Files** → **General** → 3402- Lab 3.mp4

I'm not here — read this first

Watch the Lab 3 video · 20 min

Or: **Teams** → **Files** → **General** → 3402- Lab 3.mp4

- ▶ **Today's session is that video.** No live Q&A; bring questions to Lab 4
- ▶ **Nothing to turn in, and nothing to set up.** Today is background
- ▶ Lab 3's exercises follow **Lectures 6–7** — random numbers, mean and variance, histograms, functional programming, then Monte Carlo. **Exercises 1–13, due 9/18**

I'm not here — read this first

Watch the Lab 3 video · 20 min

Or: **Teams** → **Files** → **General** → 3402- Lab 3.mp4

- ▶ **Today's session is that video.** No live Q&A; bring questions to Lab 4
- ▶ **Nothing to turn in, and nothing to set up.** Today is background
- ▶ Lab 3's exercises follow **Lectures 6–7** — random numbers, mean and variance, histograms, functional programming, then Monte Carlo. **Exercises 1–13, due 9/18**

Due dates

- ▶ **Lab 2 — Friday 9/11**, submitted *in* the Lab 4 session
- ▶ **Lab 3 — Friday 9/18**

What the video covers, and where it's now different

The video is from an earlier offering. Everything it says about the **command line** still stands. Four things have moved on:

- ▶ It says *“next week I’ll talk to you about git”* — that material is now **written down**, in this deck and in the setup guide, rather than lectured
- ▶ **Authentication is new.** GitHub passwords stopped working; the video predates that. Use `gh auth login`
- ▶ The repository is `DATA3402.Fall.2026`. Any other name you hear or read is from a past semester
- ▶ It notes a date on the slide that hadn’t been updated. Ignore dates in the video — the ones on the previous slide are correct

What I'm asking of you this week

Just this: get comfortable enough on the command line to move around and copy files.

What I'm asking of you this week

Just this: get comfortable enough on the command line to move around and copy files.

I am not asking you to set up GitHub today. We do that together in Lab 4, on Friday 9/11, from scratch. **You will not be behind if you arrive having done nothing.**

...and if you want to look ahead

Git — what it is, the vocabulary, and how this course's remotes are wired — is covered live in Lab 4, not here. Nothing to read ahead of time.

...and if you want to look ahead

Git — what it is, the vocabulary, and how this course's remotes are wired — is covered live in Lab 4, not here. Nothing to read ahead of time.

If you'd like to try something early anyway, try **GitHub authentication**:

- ▶ Instructions are in the **lab notebook**, Labs/Lab.3/Lab.3.ipynb
- ▶ It's the piece most likely to go wrong on an individual machine, so having it working makes next week quick for you

...and if you want to look ahead

Git — what it is, the vocabulary, and how this course's remotes are wired — is covered live in Lab 4, not here. Nothing to read ahead of time.

If you'd like to try something early anyway, try **GitHub authentication**:

- ▶ Instructions are in the **lab notebook**, Labs/Lab.3/Lab.3.ipynb
- ▶ It's the piece most likely to go wrong on an individual machine, so having it working makes next week quick for you

If you try it and it doesn't work, stop and bring it to Lab 4. Debugging together is what that session is for. Don't lose an evening to it alone.

The command line

Why bother

- ▶ You've interacted with computers through **graphical metaphors** — icons, windows, drag and drop. They're easy because they hide the machine
- ▶ Development and data work need you **closer to how the machine actually works**
- ▶ Some things simply cannot be done through a GUI

Why bother

- ▶ You've interacted with computers through **graphical metaphors** — icons, windows, drag and drop. They're easy because they hide the machine
- ▶ Development and data work need you **closer to how the machine actually works**
- ▶ Some things simply cannot be done through a GUI
- ▶ And yes, you can ask an AI for the command — but if you don't understand the answer, you won't know whether it did the right thing, or how to fix it when it doesn't

The shell

- ▶ Without a GUI, the interface to a machine is a **command-line shell**
- ▶ It shows a **prompt**, which tells you where in the filesystem you are
- ▶ You type a command, press enter, it runs

The shell

- ▶ Without a GUI, the interface to a machine is a **command-line shell**
- ▶ It shows a **prompt**, which tells you where in the filesystem you are
- ▶ You type a command, press enter, it runs
- ▶ Mostly the shell just **runs other programs** — you type the program's name
 - ▶ so it has to know *where to look*: that's the **PATH** environment variable
- ▶ In Unix almost everything is a program: `ls`, `rm`, `mkdir`, `rmdir`, ...
- ▶ A few things are built into the shell itself, like `cd`
- ▶ **Environment variables** are a map of name → value that programs read to find things

Moving around

- ▶ `pwd` — print working directory: where am I?
- ▶ `ls` — list what's here · `ls -l` — with sizes, dates, permissions
- ▶ `cd <dir>` — go in · `cd ..` — go up one · `cd ../..` — up two · `cd -` — back where I was

Moving around

- ▶ `pwd` — print working directory: where am I?
- ▶ `ls` — list what's here · `ls -l` — with sizes, dates, permissions
- ▶ `cd <dir>` — go in · `cd ..` — go up one · `cd ../..` — up two · `cd -` — back where I was

Files

- ▶ `mkdir <dir>` — make a directory · `rmdir <dir>` — remove an empty one
- ▶ `cp <from> <to>` — copy · `mv <from> <to>` — move or rename · `rm <file>` — delete
- ▶ `man <command>` — the manual. Space to scroll, q to quit

This is the same navigating you do in Finder or Explorer — just typed.

How you'll actually write code

- ▶ **Most basic:** a text editor for the code, comments for the explanation, a shell to run it
- ▶ **IDEs** put editing, running and debugging in one place
- ▶ **Notebooks** combine text, code and output in one document — which is why this course lives in them

Where to look

This week

- ▶ Labs/Lab.3/Lab.3.ipynb — the command-line reference, the **GitHub authentication instructions**, and the exercises due 9/18
 - ▶ the last three go past the video: inverse-transform sampling, the $1/\sqrt{N}$ error bar on your own integral, and percentiles from a cumulative sum
 - ▶ all three are in **Chapter 3**, which is released — read that, not the video, for them

This week

- ▶ Labs/Lab.3/Lab.3.ipynb — the command-line reference, the **GitHub authentication instructions**, and the exercises due 9/18
 - ▶ the last three go past the video: inverse-transform sampling, the $1/\sqrt{N}$ error bar on your own integral, and percentiles from a cumulative sum
 - ▶ all three are in **Chapter 3**, which is released — read that, not the video, for them

Next week — the git lecture

- ▶ **Lab 4** — git itself: what it is, the vocabulary, how this course's remotes are wired, and your first push. Live, with time for questions
- ▶ Labs/Lab.4/Lab.4.ipynb and that week's deck — the concepts and the steps, in order
- ▶ Labs/GitHub-Setup.pdf — the same ground in more detail, with an error table
- ▶ A **merge conflict** on `git pull` means a file we handed out was edited in place. Nothing is lost — Labs/GitHub-Setup.pdf has the steps that keep your work and get you pulling again